Iterated Prisoner's Dilemma Machine Learning Analysis

COMP-3710 Artificial Intelligence Concepts

Dr. Robin Gras

March 4th, 2026

Hadiyah Arif
110076280
arifh@uwindsor.ca

Aakanksha Mandal
110189000
mandal5@uwindsor.ca

Linh Khanh Nguyen
110119278
nguyen4m@uwindsor.ca

1. Abstract

This project investigates how cooperation develops in the Iterated Prisoner's Dilemma (IPD) using both traditional game theory and modern AI techniques. We began by building a simulation engine and implementing six human-designed strategies as baselines, including Always Cooperate, Always Defect, Random, and three variations of the Tit-for-Tat strategy. After establishing these benchmarks, we applied a Genetic Algorithm to evolve new strategies represented as bitstrings in a lookup table. Finally, we used machine learning models to analyze the logic of the best-performing strategies. Experimental results indicate that evolved strategies achieved higher average scores relative to traditional baselines, illustrating the effectiveness of evolutionary optimization. Our machine learning analysis suggests that effective strategies combine cooperation with rapid punishment and selective forgiveness, highlighting how adaptive learning processes can promote the emergence of cooperative behaviour in repeated interactions. This report explains our simulator design, the performance of our baseline strategies, and the findings from our evolutionary and machine learning analysis.

2. Introduction

The Prisoner's Dilemma is a classic problem in game theory because it shows why two people might not cooperate even if it is better for both of them. In a single round, the safest bet is to defect, which means betraying the other person. But when you play the game many times in a row, which is the iterated version, things change. You have to think about your reputation and how the other person will react to what you do.

For this project, we wanted to find the most effective way to handle this situation. We started by implementing and testing six standard strategies to understand the basic mechanics and set a benchmark for our work. Once we had those baselines, we used an algorithm to see if it could evolve new strategies that were even better on their own. We split the work into three stages. First, we built a tournament simulator and the baseline strategies. Second, we ran a Genetic Algorithm to evolve new strategies. Third, we used Machine Learning to figure out the rules the AI actually followed. Our goal was to see if the AI would naturally start cooperating or if it would find ways to cheat.

3. Relevant Literature Review

We started with Robert Axelrod's work from the 1980s. He held a tournament where computer programs played the IPD against each other. The simplest strategy, Tit-for-Tat, ended up winning. This proved that being nice but firm is a good approach. We also looked at John Miller's research, which was the first to use Genetic Algorithms to represent these strategies as bitstrings. This is where we got the idea for our lookup table that maps 3-round histories to specific moves.

To understand how memory affects the game, we read Kristian Lindgren's papers. He found that having more memory lets strategies be more forgiving, which stops a single mistake from causing a permanent war between players. Finally, we looked at how modern studies

use Machine Learning, like the research by Wang in 2017, to classify these evolved strategies. This background helped us decide on our memory depths and gave us a baseline to compare our own results against.

4. Experimental Setup and Methodology

4.1 Payoff System

We used the standard payoff system for our simulator. It gives 3 points for mutual cooperation, 1 point for mutual defection, and 5 points for betraying someone who cooperated. Each match lasted for 200 rounds, so the strategies had enough time to interact.

4.2 The Simulator and Baseline Strategies

The first part of our project was building a Python engine that handles the matches and tournaments. The engine keeps track of every move so that strategies can look back at the history to decide what to do next. We coded six specific baseline strategies to test the environment and provide a clear point of comparison:

- **Always Cooperate (ALLC):** A naive strategy that cooperates every round.
- **Always Defect (ALLD):** A selfish strategy that always betrays the opponent.
- **Random (RAND):** A strategy that flips a coin to decide its move.
- **Tit-for-Tat (TFT):** Starts by cooperating and then copies the opponent's previous move.
- **Tit-for-Two-Tats (TF2T):** A more forgiving version of TFT that only defects if the opponent defects twice in a row.
- **Suspicious Tit-for-Tat (STFT):** Similar to TFT, but starts the match with a defection.

4.3 Strategy Encoding

Strategies were encoded as lookup tables parameterized by memory depth *d*. Since each round has four possible outcomes (CC, CD, DC, DD), the chromosome length equals: 4^d. Each entry specifies whether to cooperate (0) or defect (1) for that historical state. For example, depth 3 corresponds to 64 bits, while depth 5 corresponds to 1024 bits. This representation enables evolutionary optimization over flexible memory capacities.

4.4 Genetic Algorithm Implementation

We implemented a Genetic Algorithm (GA) to evolve Iterated Prisoner's Dilemma strategies represented as lookup-table chromosomes with depth-dependent length. For a memory depth *d*, each chromosome has length 4^d, where each position corresponds to one possible interaction history of length *d*. Each entry stores a binary action (0 = Cooperate, 1 = Defect).

The main mutation-rate sweep experiments reported in this section were conducted using depth 3 (64-bit chromosomes). A separate depth comparison experiment (3 vs 5) is reported in Section 5.2.

Fitness was defined as the average total score achieved by a candidate strategy over 200-round matches against a fixed opponent pool consisting of six baseline strategies (ALLC, ALLD, RAND, TFT, TF2T, STFT) plus ten randomly generated lookup-table strategies.

For the primary GA configuration, we used:

- Population size = 80
- Generations = 200
- Crossover rate = 0.8
- Elite size = 2
- Rounds per matchup = 200
- Seed = 42

We evaluated mutation rates of 0.0005, 0.001, and 0.005. The best overall result occurred at a mutation rate of 0.001, achieving a peak fitness of 708.8125.

To quantify convergence, we analyzed the per-generation fitness history. Best fitness increased rapidly during early generations (reaching approximately 702 by generations 35–40). After roughly 60 generations, improvements became incremental and noisy, indicating entry into a plateau region. The final new global-best improvement occurred at generation 123 (fitness = 708.8125), after which performance fluctuated within a narrow range through generation 200.

4.5 Machine Learning Classification

To examine the behavioural patterns of evolved strategies, supervised machine learning models were applied to the lookup-table representations produced during the optimization process. Each strategy with a memory depth of three was encoded as a 64-bit feature vector, where each bit represents the action taken under a specific history of previous interactions. Strategies were evaluated against a fixed opponent pool and their average scores were recorded. A binary classification label was then assigned to identify high-performing strategies, defined as those within the top 20% of scores.

Three classification models were trained: Logistic Regression, Decision Tree, and Random Forest. Logistic Regression served as a linear baseline model, while the tree-based models were used to capture nonlinear relationships between historical decision patterns and strategy performance. The dataset was divided into training and testing sets using an 80/20 split with a fixed random seed to ensure reproducibility. Model performance was evaluated using accuracy and F1-score, with ROC-AUC used when probability estimates were available.

The Decision Tree and Random Forest models achieved better performance than Logistic Regression, indicating that successful strategies depend on combinations of previous responses rather than simple independent decision rules. In particular, the Decision Tree model allowed us to interpret how specific lookup-table entries influenced strategy success. The model was able to reliably classify high-performing strategies based on their bitstring representations, demonstrating that evolved strategies contain recognizable structural

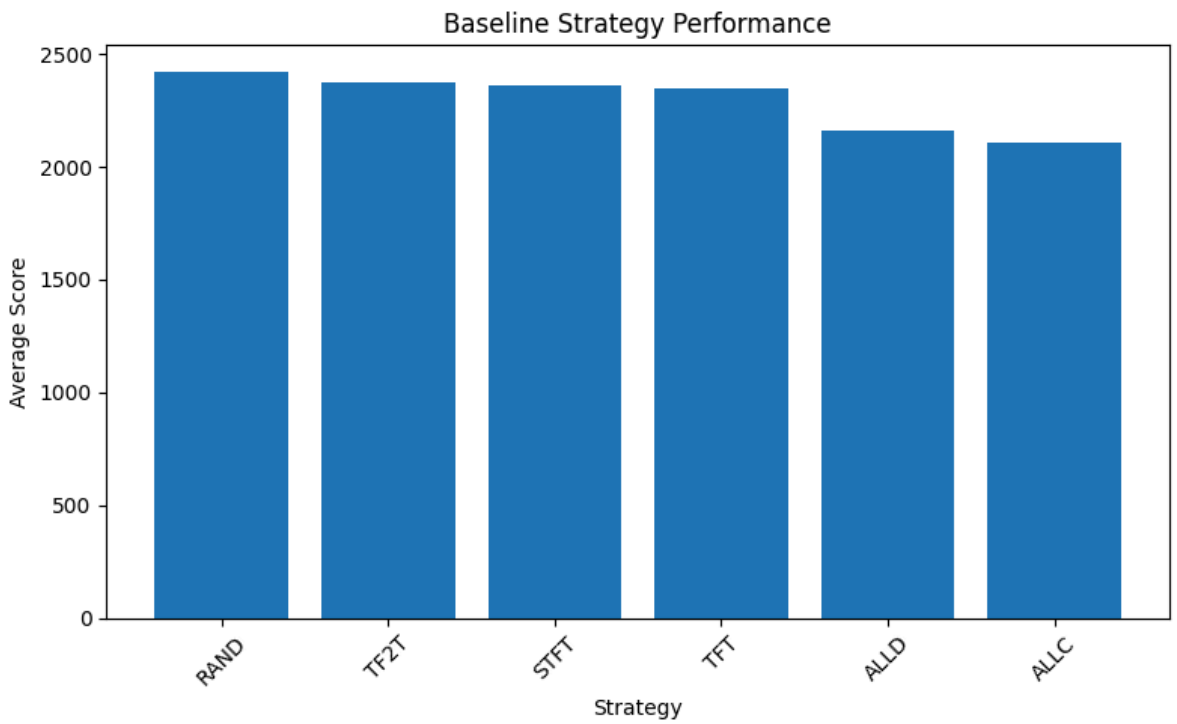
patterns. These patterns correspond to well-known behavioural principles in the Iterated Prisoner's Dilemma, such as maintaining cooperation after mutual cooperation and retaliating after opponent defection.

5. Discussion of Findings

5.1 Baseline Performance

When we tested our simulator using the six baseline strategies, we observed clear differences in performance based on the population composition. In the Standard Tournament, the Random strategy (RAND) achieved the highest total score (2400), suggesting that unpredictability can exploit more 'forgiving' environments. The Tit-for-Tat variants—TFT (2341), TF2T (2370), and STFT (2351)—performed consistently well, showing that reciprocity-based strategies remain robust in balanced interactions. Always Defect (ALLD) scored lower (2188) due to mutual defection penalties, while Always Cooperate (ALLC) scored the lowest (2127) as it was repeatedly exploited.

However, when we introduced a Stress Test by adding more defective opponents, the dynamics shifted. In this more aggressive environment, reciprocity became the dominant trait, with TF2T (2992) and TFT (2957) significantly outperforming RAND (2720). This shift proves that while random behaviour can be parasitic in cooperative pools, structured reciprocity is the only evolutionarily stable strategy as competition increases. Overall, these results validate that the simulation engine behaves as expected and provides a solid benchmark for the Genetic Algorithm.



5.2 Results of the Evolution

The Genetic Algorithm consistently improved strategy fitness over time, with rapid gains in early generations followed by convergence to a plateau. In the depth-3 best run (seed 42), best fitness increased from 540.38 at generation 0 to approximately 700 by generation 30–40, indicating great early improvement through exploration and selection pressure. After this point, improvements became smaller and more incremental, with fitness fluctuating within a narrow high-performance band. The final global-best improvement occurred at generation 123, reaching a peak fitness of 708.8125, after which performance largely stabilized through generation 200. This pattern reflects typical evolutionary behaviour: fast early optimization followed by fine-tuning around a near-optimal region.

The corresponding evolved bitstring strategy was:
1000011110111001010011101101111110111110111111101111011110101101

To understand what behavioural traits the algorithm evolved, we evaluated the GA-best strategy head-to-head against the baseline strategies over 200 rounds. The evolved strategy was not purely cooperative; rather, it exhibited opportunistic behaviour. It strongly exploited unconditional cooperation, scoring 992–12 against ALLC, and achieved large advantages against forgiving opponents such as TF2T (664–334) and RAND. Against reciprocal strategies such as TFT and STFT, the evolved strategy tied (534–534), indicating that it avoided sustained aggression that would trigger retaliation. Against ALLD, the strategy slightly underperformed (196–216), reflecting mutual defection dynamics. Overall, the evolved strategy combined cooperation in stable states with targeted defection in exploitable situations, suggesting adaptive mixed behaviour rather than unconditional cooperation.
We also compared memory depths 3 and 5 to evaluate the impact of increased historical information.

Results from the tagged depth sweep (ga_depth_sweep_summary_d3_vs_d5.csv) show clear differences:

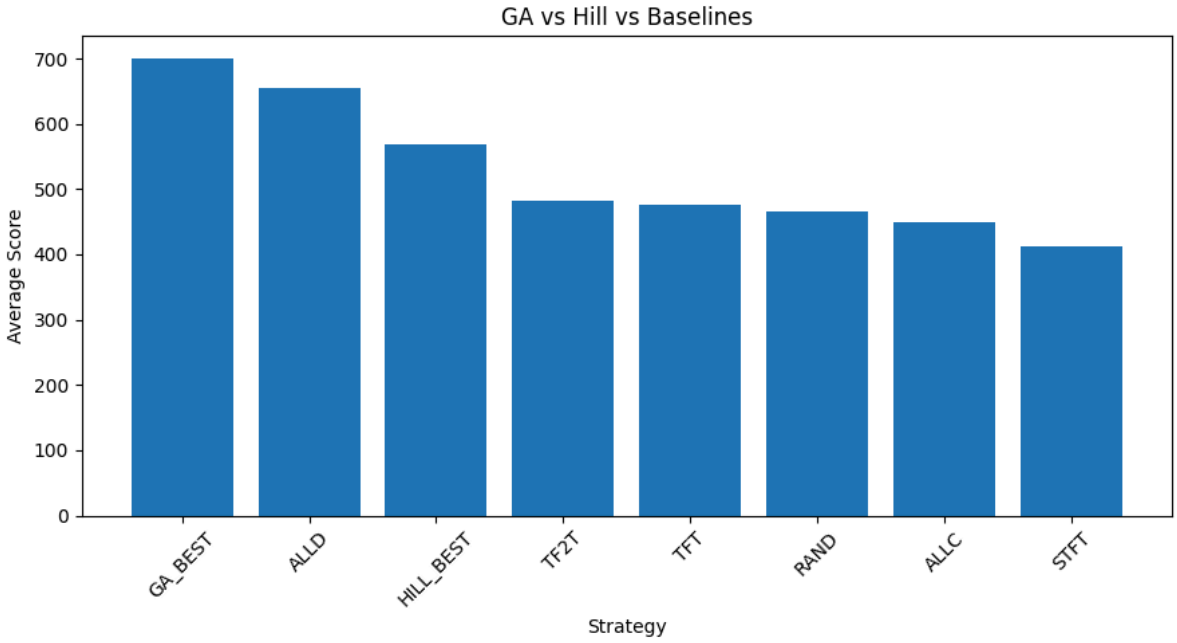
Depth 3
- Mean best fitness: 697.92
- Mean final average fitness: 688.70
- Mean generation reaching 99% of the final best: 56.33

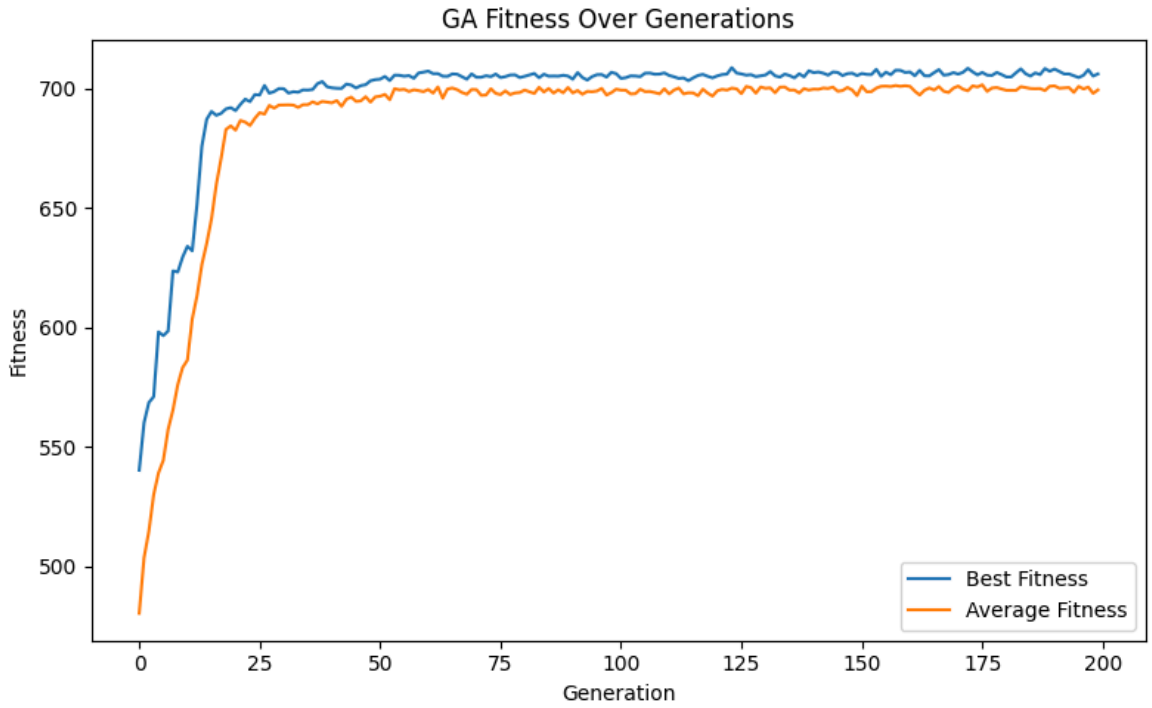
Depth 5
- Mean best fitness: 740.40
- Mean final average fitness: 732.99
- Mean generation reaching 99% of the final best: 163.67

Depth 5 produced stronger final solutions, achieving significantly higher average and peak fitness values. However, it required substantially more generations to approach its optimal performance, reflecting the increased search complexity of a larger strategy space (1024-bit

representation versus 64-bit at depth 3). In contrast, depth 3 converged more quickly but to slightly weaker solutions.

Finally, we implemented Hill Climbing as a comparison method. Hill Climbing improved fitness rapidly in early iterations but converged to a lower best fitness of 573.5 after 5000 iterations, indicating convergence to a local optimum. Compared to Hill Climbing, the Genetic Algorithm demonstrated superior performance and greater ability to explore the high-dimensional strategy space. This supports the conclusion that population-based evolutionary search with crossover and mutation is more effective than purely local bit-flip search for evolving complex IPD strategies.





5.3 Machine Learning Analysis of Evolved Rules

The Decision Tree model was used to identify the most influential features and extract interpretable decision rules learned during classification. This analysis was conducted to better understand the behavioural logic behind successful evolved strategies. Because each feature corresponds to a specific lookup-table response to a three-round interaction history, the learned rules reveal how particular historical states influence strategic decisions.

The extracted rules showed that high-performing strategies followed consistent behavioural patterns. In particular, strategies predicted to perform well almost always cooperated following mutual cooperation states (CC). Maintaining cooperation after CC helps stabilize mutually beneficial interactions and maximizes long-term rewards. The model also indicated that successful strategies responded quickly to opponent defection by defecting in the following round. This immediate retaliation prevents repeated exploitation while maintaining strategic fairness. Such behaviour closely resembles the principles of Tit-for-Tat strategies, where cooperation continues as long as both players cooperate but defection is punished when it occurs.

Another important pattern identified by the Decision Tree was the avoidance of prolonged mutual defection. Strategies that continued defecting indefinitely after a conflict were less likely to be classified as high-performing. Instead, successful strategies often returned to cooperation after a short punishment phase. This behaviour introduces forgiveness into the interaction, allowing cooperation to recover after mistakes or temporary conflicts.

Overall, the machine learning analysis suggests that evolved strategies balance three key behavioural principles: cooperation, retaliation, and forgiveness. Rather than relying solely on purely cooperative or purely defective behaviour, high-performing strategies adapt their responses based on previous interactions. These findings demonstrate that evolutionary optimization naturally discovers strategic behaviours consistent with established theoretical models of cooperation in the Iterated Prisoner's Dilemma.

6. Concluding Remarks

This project examined the development of collaboration in the Iterated Prisoner's Dilemma using a combination of simulation, evolutionary optimization, and machine learning analysis. Baseline methods served as a standard for comparison, whereas the Genetic Algorithm effectively developed lookup-table strategies that attained above average performance against a static opponent pool. These results suggest that evolutionary methods are better at identifying solutions that work than traditional handmade ones.

It was easy to figure out how the behaviours changed when we used machine learning. Classification models might be able to guess which methods will work best by looking at the patterns in their choices. This illustrates that great solutions include some parts that don't change, not behaviours that are impossible to predict. The Decision Tree principles claimed that the best solutions are those that mix working together with quick punishment and kind

forgiveness. This stops people from utilizing each other and makes them want to work together. These behavioural traits align strongly with prior theoretical insights regarding reciprocal cooperation in repeated games.

Using evolutionary computation and machine learning that can be easily combined can help you make better strategic decisions and explain why they work. The results demonstrate that cooperative behaviour can organically develop through adaptive processes of learning when agents react conditionally to previous interactions. Future studies may explore enhanced memory capacities, a wider range of negative groups, or varied methods for instruction to more effectively analyze the dynamics of cooperation in complicated situations.

6.1 Future Work

Several extensions could further improve the performance and understanding of evolved strategies in the Iterated Prisoner's Dilemma.

Although this project compared memory depths 3 and 5, further investigation across additional depths (e.g., 4, 6, or higher) and a broader range of random seeds would provide a more complete understanding of the relationship between strategy complexity and optimization difficulty. While depth 5 achieved stronger final fitness values, it required substantially more generations to converge. Future studies could examine whether performance continues improving with increasing depth or eventually plateaus due to the rapidly expanding search space.

Beyond memory depth, the Genetic Algorithm itself could be enhanced. Adaptive mutation rates, larger population sizes, longer evolutionary runs, or alternative selection mechanisms may improve exploration of the strategy space. More advanced evolutionary approaches such as tournament selection, multi-objective optimization, or co-evolution—where candidate strategies evolve against one another dynamically rather than against a fixed opponent pool—could better simulate competitive environments and potentially produce more robust solutions.

The evaluation framework could also be expanded. In this project, a fixed opponent pool was used to ensure reproducibility and fair comparison. However, real strategic environments are dynamic. Allowing the opponent population to evolve simultaneously or introducing stochastic noise into interactions would provide a more realistic test of long-term strategy stability and adaptability.

Finally, while this study used supervised machine learning models to interpret evolved lookup-table strategies, future work could explore more expressive models. Reinforcement learning agents or neural-network–based policies could learn strategies directly from interaction data rather than from predefined lookup structures. Such approaches may uncover more complex behavioural patterns and offer deeper insight into the mechanisms underlying the emergence of cooperation in repeated interactions.

7. References

- Axelrod, R. (1984). *The Evolution of Cooperation*. Basic Books.
- Axelrod, R. (1987). *The Evolution of Strategies in the Iterated Prisoner's Dilemma*.
- Lindgren, K. (1991). *Evolutionary Phenomena in Simple Dynamics*. Physica D.
- Wang, K. (2017). *Iterated Prisoner's Dilemma with Reinforcement Learning*. Stanford University.

8. Appendix

8.1 Project Repository and Code Access

All source code, configuration files, and documentation for this project are hosted on GitHub. The repository includes the modular Python engine, baseline strategy implementations, genetic algorithm scripts, and the datasets used for machine learning analysis. The full project history and contribution logs can be viewed at the following link:

- **GitHub Repository:** https://github.com/Hadiyahh/Iterated-Prisoners-Dilemma-ML

8.2 Participation and Contributions

While this project was a collaborative effort in terms of high-level logic and strategy, we divided the technical and writing responsibilities to ensure all project goals were met. The specific contributions were as follows:

- **Aakanksha**
    - Established the initial GitHub repository structure setup.
    - Developed the core Python simulation engine and payoff matrix logic.
    - Implemented the six baseline strategies: ALLC, ALLD, RAND, TFT, TF2T, and STFT.
    - Ran the baseline tournaments and a stress test with extra defectors.
    - Authored the following report sections:
        - 1. Abstract
        - 2. Introduction
        - 3. Relevant Literature Review
        - 4. Experimental Setup and Methodology
            - 4.1 Payoff System
            - 4.2 Simulator and Baseline Strategy
            - 4.3 Strategy Encoding
        - 5. Discussion of Findings
            - 5.1 Baseline Performance
        - 7. References
        - 8. Appendix
            - 8.1 Project Repository and Code Access
            - 8.2 Participation and Contributions
            - 8.3 Use of AI Tools

- **Hadiyah:**
  Led the design, implementation, and evaluation of the optimization components of the project. Specific contributions included:
  - Designed and implemented the Genetic Algorithm framework to evolve lookup-table strategies represented as depth-dependent chromosomes of length 4^d, with main experiments conducted at depth 3 and additional comparison experiments at depths 3 and 5.
  - Defined the GA configuration, including population size, mutation rate, crossover rate, elite selection, and generation count.
  - Implemented experiment runners and automated evaluation pipelines to train, evaluate, and compare evolved strategies against baseline opponents.
  - Generated and analyzed fitness history, best-strategy outputs, and comparative performance datasets across multiple GA runs.
  - Implemented Hill Climbing as a secondary optimization method and compared its performance against the Genetic Algorithm.
  - Saved and structured experimental results into CSV datasets for further analysis and machine learning classification.
  - Authored and documented the following report sections:
    - 4.4 Genetic Algorithm Implementation
    - 5.2 Results of the Evolution
      - Concluding Remarks
    - 6.1 Future Work
    - Appendix sections related to optimization and contributions
      - 8.2 Participation and Contributions

- **Linh:**
  - Designed and executed experimental evaluations using a fixed opponent pool to ensure consistent and reproducible results.
  - Generated experimental result tables, including baseline performance, optimizer comparisons, and memory-depth evaluations.
  - Produced all visualization figures, including baseline performance charts, genetic algorithm fitness curves, comparison plots, and memory analysis graphs.
  - Constructed the machine learning dataset from evolved and randomly generated strategies using 64-bit lookup-table representations.
  - Implemented and evaluated machine learning models (Logistic Regression, Decision Tree, and Random Forest) to classify high-performing strategies.
  - Extracted interpretable decision rules and analyzed behavioural patterns identified by the machine learning models.
  - Authored the following report sections:
    - 4. Experimental Setup and Methodology
      - 4.5 Machine Learning Classification
    - 5. Discussion of Findings
      - 5.3 Machine Learning Analysis of Evolved Rules

- 6. Concluding Remarks
- 8. Appendix
    - 8.2 Participation and Contributions

8.3 Use of AI Tools

We used Gemini to assist in organizing the code structure and to locate and summarize foundational academic papers for the literature review. The tool was also used to help refine the formatting of this report to ensure academic clarity and consistent section balancing.